Classification with probabilities

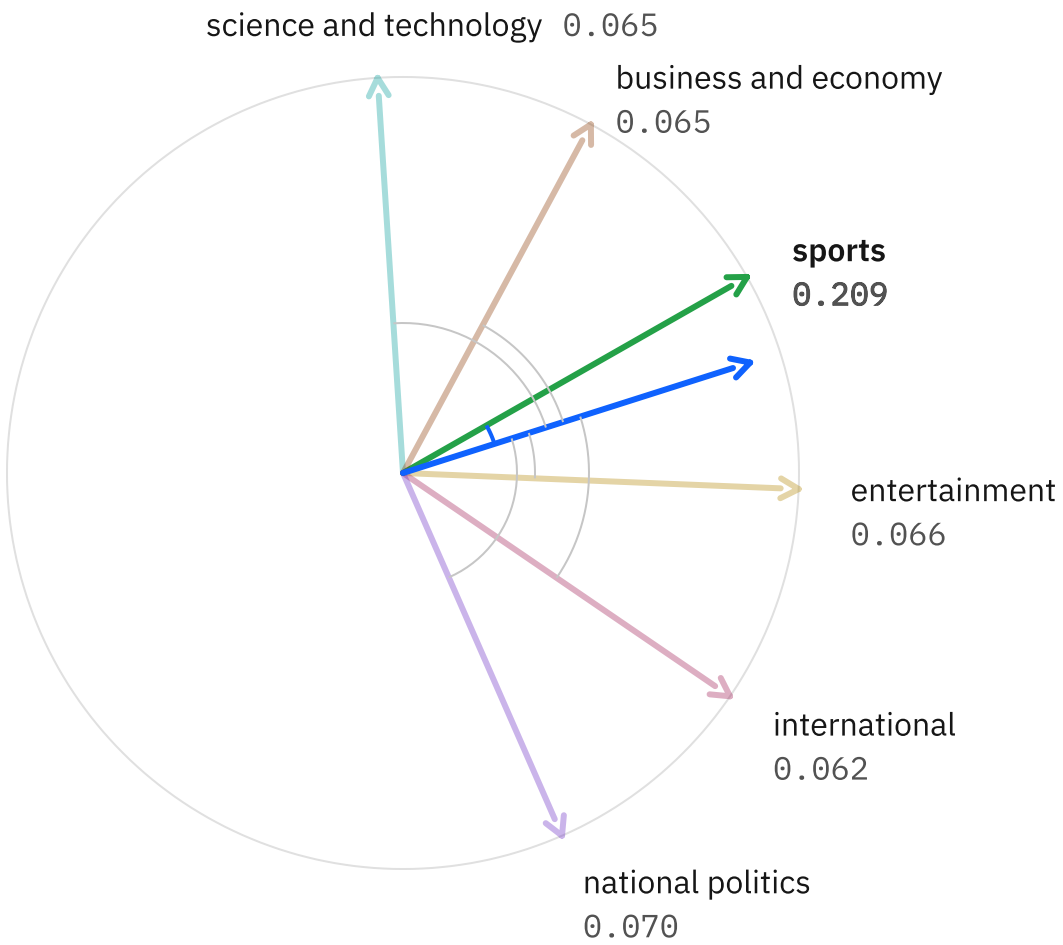
Class 2 of 24: Natural Language Processing

CSE 4811 / DS 4213 (Fall 2026)

Muhammad Anwarul Azim
Lecturer, Dept. of CSE
United International University

Recap

A new article: its cosine with each average. The topic with the largest cosine similarity is selected.



a new article

প্রথম ও শেষ কথা জয়: মাশরাফি
বিশ্বকাপে নিজেদের পঞ্চম ম্যাচে সোমবার
...

nearest average: **sports (right)**

The six cosines are computed by the class 1 model; the angles are a sketch.

How good was our classification?

Consider one topic: Sports

518 news classified as sports



497 right — 21 wrong

The 21 wrong were really

national politics	3
international	4
business and economy	2
entertainment	6
science and technology	6

666 texts really sports



497 found — 169 missed

The 169 missed were called

national politics	15
international	27
business and economy	76
entertainment	49
science and technology	2

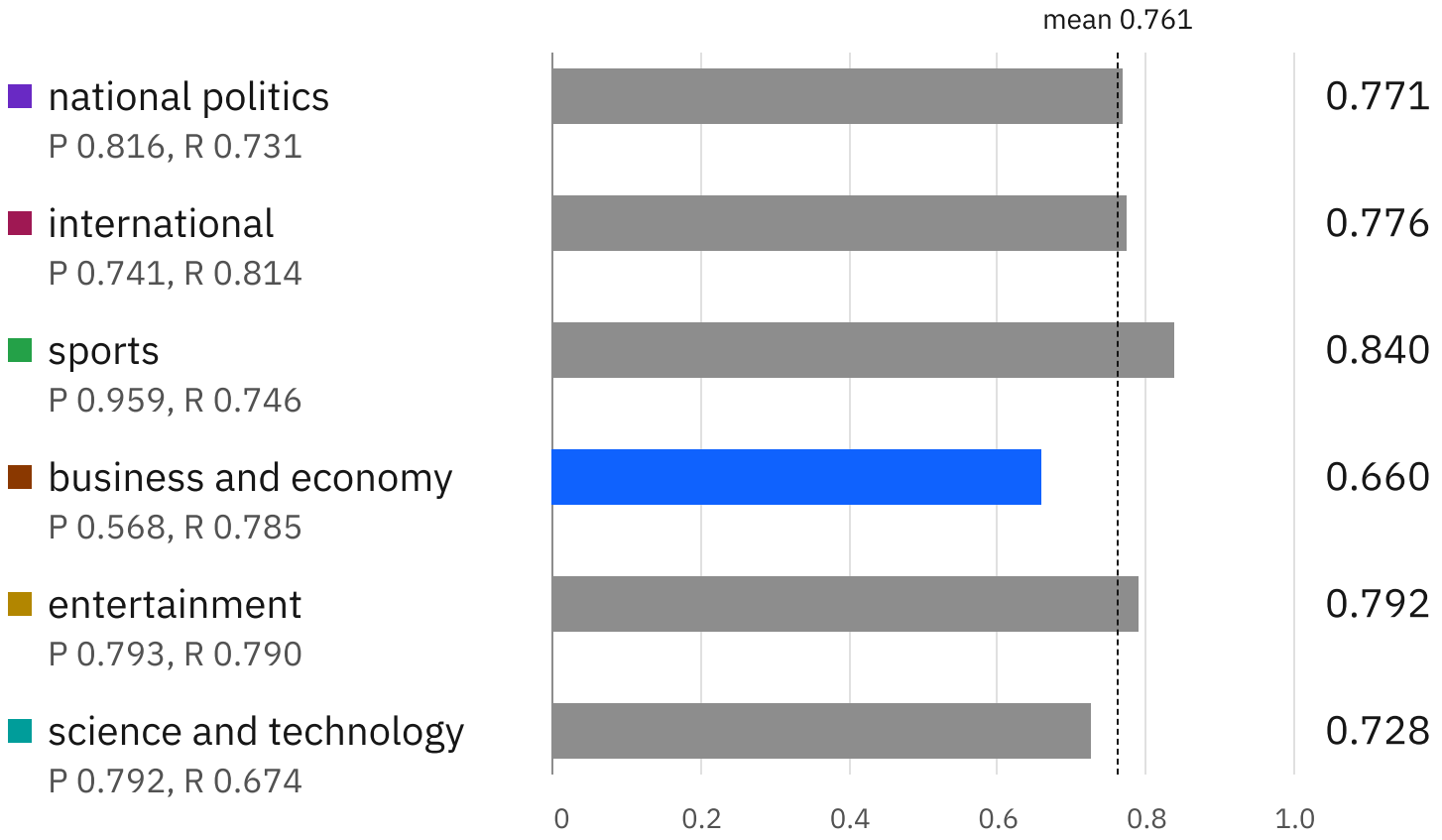
$$\text{Precision} = \frac{497}{518} = 0.959$$

$$\text{Recall} = \frac{497}{666} = 0.746$$

$$\text{F1} = \frac{2 \times 0.959 \times 0.746}{0.959 + 0.746} = 0.840$$

Why macro-F1?

F1 for each topic. Nearest topic average on TF-IDF, 4,000 dev texts.



0.761

macro-F1, the mean of the six

0.757

accuracy, the share of texts right. Our dev set is balanced, so accuracy and macro-F1 are close.

Macro-F1 gives every topic the same weight.

Our score

Dev set: 4,000 news articles that were not used to compute the six topic averages

0.761

macro-F1



973 of 4,000 wrong

Training set: the 40,000 news articles used to compute the six topic averages

0.782

macro-F1



8,848 of 40,000 wrong

It also misclassifies 22 % of the texts its averages were computed from.

Part 1: How to improve

What the nearest topic average cannot do, and what a learning model needs.

Two major limitations

1

It ignores word order

```
Bangladesh beat India
India beat Bangladesh
```

the same vector | `bangladesh 1` | `beat 1` | `india 1`

2

It can't learn

8,848 of 40,000 wrong.

Mistakes are normal. A learning system must learn from its mistakes.

To learn from mistakes, we need

a loss function
how wrong the model is

→

a way to adjust the model
so that the loss falls

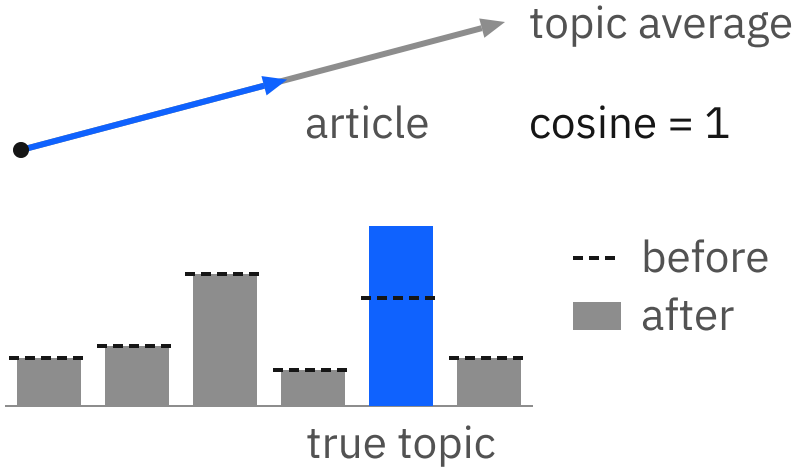
Today: the loss

Why cosine cannot be the loss

Suppose we train the average model to make the cosine larger.

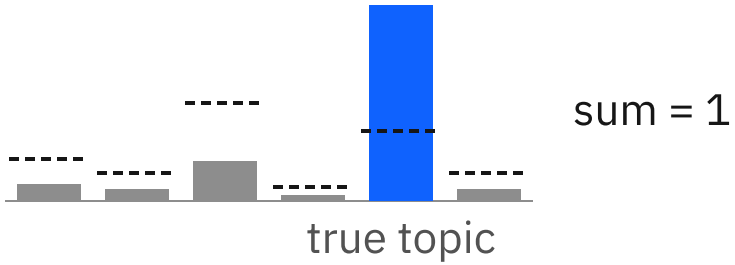
1. A cosine of 1 is not the goal
A cosine of 1 means the article has the same mix of words as its topic average. We never wanted that.

2. Cosine does not move away from the rivals
A learning model should move toward the true topic and away from the false topics. Cosine scores each topic on its own. A larger cosine for the true topic leaves the other cosines as they are.



We need probabilities
Each news item belongs to one topic. Our goal is to be fully confident of that topic: $p(\text{true topic} \mid \text{article}) = 1$.

The six probabilities add up to 1. When the true topic's probability rises, the probabilities of the other topics fall.



1. Probability → 2. Loss → 3. Adjustment

Part 2: Computing probabilities

From word counts to six probabilities that add up to 1.

Bayes' rule

$$p(k \mid \text{text}) = \frac{p(\text{text} \mid k)\ p(k)}{\sum_j p(\text{text} \mid j)\, p(j)}$$

Likelihood	$p(\text{text} \mid k)$	how likely topic k is to write this text
Prior	$p(k)$	how common topic k is
Denominator	$\sum_j p(\text{text} \mid j)\, p(j)$	the sum of the six numerators, so the six results add up to 1
Posterior	$p(k \mid \text{text})$	what we want

We need two things: the likelihood and the prior.

A table of word probabilities

One row per topic, one column per word: 6 rows, V columns.

Toy example: two topics, three words.

$$p(w \mid k) = \frac{\text{count of } w \text{ in topic } k}{\text{total words in topic } k}$$

A cell is the chance that a word drawn at random from this topic's articles is w.

This is the unigram model: one word at a time.

	ম্যাচ match	তারকা star	ছবি film	Total
sports	0.6	0.3	0.1	1.0
entertainment	0.1	0.4	0.5	1.0

Each count divided by its row total

Prior and likelihood

Prior

$$p(k) = \frac{n_k}{N}$$ n_k training texts in topic k, N in all

Toy example: both topics equally common, so 0.5 and 0.5.

C1 training set	Texts	$p(k)$
international	6,809	0.170
national politics	6,809	0.170
business and economy	6,807	0.170
entertainment	6,806	0.170
sports	6,550	0.164
science and technology	6,219	0.155
All	40,000	1.000

Likelihood

A new headline: বড় তারকা, নতুন ছবি (big star, new film).

Only তারকা and ছবি are in the toy vocabulary; the other words are skipped.

$p(w \mid k)$	তারকা	ছবি	Likelihood
sports	0.3	0.1	0.3 × 0.1 = **0.03**
entertainment	0.4	0.5	0.4 × 0.5 = **0.20**

The two do not add up to 1. They answer the reverse question: how likely each topic is to write this text.

Bayes' rule in action

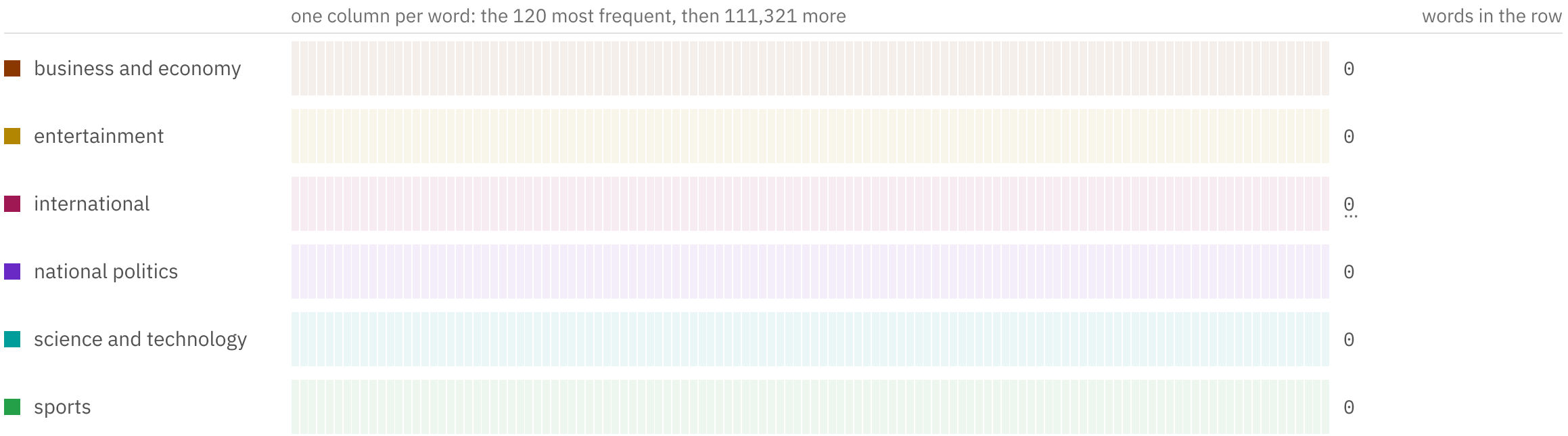
$$p(k \mid \text{text}) = \frac{p(\text{text} \mid k)\, p(k)}{\sum_j p(\text{text} \mid j)\, p(j)}$$

Topic	Likelihood	Prior	Likelihood × prior	Posterior
sports	0.03	0.5	0.015	0.015 / 0.115 = **0.13**
entertainment	0.20	0.5	0.100	0.100 / 0.115 = **0.87**
Sum			0.115	1.00

Two probabilities that add up to 1. The headline is entertainment with probability 0.87.

The count matrix

Six rows, one per topic. **111,441** columns, one per word. Every cell starts at 0.



Underflow, and the fix

A real article has hundreds of words, each with a probability of about 1 in 500.

n	Word	p	`float16`	`float32`	`float64`	$\sum \log p$
	Smallest positive number		6.0×10^{-8}	1.4×10^{-45}	4.9×10^{-324}	
1	`rice`	0.002	2.0×10^{-3}	2.0×10^{-3}	2.0×10^{-3}	−6.2
2	`prices`	0.002	4.0×10^{-6}	4.0×10^{-6}	4.0×10^{-6}	−12.4
3	`rose`	0.002	**0**	8.0×10^{-9}	8.0×10^{-9}	−18.6
⋮						
17	`market`	0.002	0	**0**	1.3×10^{-46}	−105.6
⋮						
120	`traders`	0.002	0	0	**0**	−745.8
⋮						
300	`week`	0.002	0	0	0	−1864.4

$$z_k = \log p(k) + \sum_i \log p(w_i \mid k)$$

- sports −0.693 − 1.204 − 2.303 = **−4.200**
- entertainment −0.693 − 0.916 − 0.693 = **−2.303**

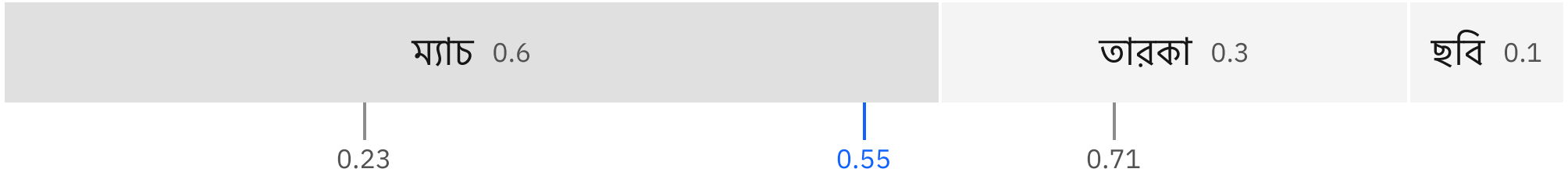
From scores to probabilities: softmax

	score z_k	subtract the largest	take $e^{\cdot}$	divide by the sum
sports	−4.200	−1.897		0.13
entertainment	−2.303	0		0.87
sum				1.00

This is called softmax. $$p(k \mid \text{text}) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Naive Bayes can write

Take the sports row. Each cell is a segment of a bar of length 1.



Draw a random number between 0 and 1. The segment it lands in gives the word.

তারকা	ম্যাচ	ম্যাচ	end

Repeat until the draw lands on an end mark.

Sampling from a Naive Bayes

Topic

- business and economy
- entertainment
- international
- national politics
- science and technology
- sports

One draw from the row

johnny	it's	প্রশ্ন	দিনে	লড়াই
0.0001	0.0006	0.0002	0.0004	0.0004



Document

sports — 15 of 15 words

3 and করে an কাটিয়ে i'm যাত্রা বাড়িয়ে she simone নেমে bay national ঠিকানা লড়াই

One table, two jobs

	ম্যাচ	তারকা	ছবি
sports	0.6	0.3	0.1
entertainment	0.1	0.4	0.5

Write

Pick a topic, then draw word after word along its row.

ম্যাচ | তারকা | ম্যাচ | ম্যাচ | ছবি

Classify

Take the words of the text, multiply along each row, then Bayes' rule.

তারকা | ছবি

sports	0.3 × 0.1 = 0.03	0.13
entertainment	0.4 × 0.5 = 0.20	**0.87**

Part 3: Constructing the loss

One number that says how wrong the model was.

probability of the true topic → loss

Loss in classification

In classification, the model gives a probability to **every topic**.

বড় তারকা, নতুন ছবি

sports	0.13	
entertainment	0.87	true topic

$$p = 0.87$$

The loss depends only on the probability of the true topic.

A loss $L(p)$ is a function of the probability p that must satisfy the following:

1. $L(1) = 0$
   No loss when the true topic gets probability 1.
2. L falls as p rises
   A larger probability gives a smaller loss.
3. L rises as p falls
   A smaller probability gives a larger loss.

> **What function can we use for the loss?**
>
> The simplest choice is $L(p) = 1 - p$, **but it has several problems.**

Cannot tell a confident error from an uncertain one

	Model A	Model B
Text 1	0.6	0.99
Text 2	0.6	0.99
Text 3	0.6	0.99
Text 4	0.6	0.02
Correct	4 of 4	3 of 4

Total loss with $1 - p$

Model A $0.4 + 0.4 + 0.4 + 0.4 = 1.60$



Model B $0.01 + 0.01 + 0.01 + 0.98 = 1.01$



Problem: with $1 - p$, Model B has a lower loss than Model A. It should be the opposite, because Model A is correct on every text.

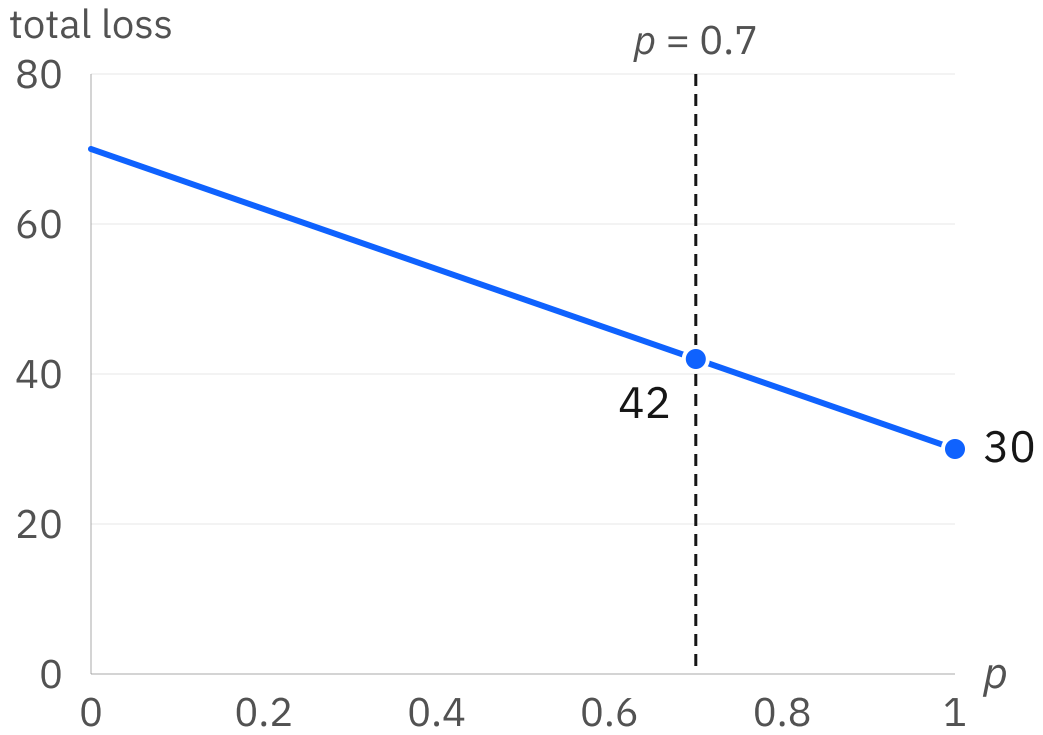
Prefers an overconfident answer to an honest one

Headlines	$p(\text{true topic})$	Loss
70 sports	p	$L(p) = 1 - p$
30 entertainment	$1 - p$	$L(1-p) = p$

$$\begin{aligned}\text{Total loss} &= 70\,L(p) + 30\,L(1-p) \\ &= 70\,(1-p) + 30\,p\end{aligned}$$

At $p = 0.7$ total loss $= 42$

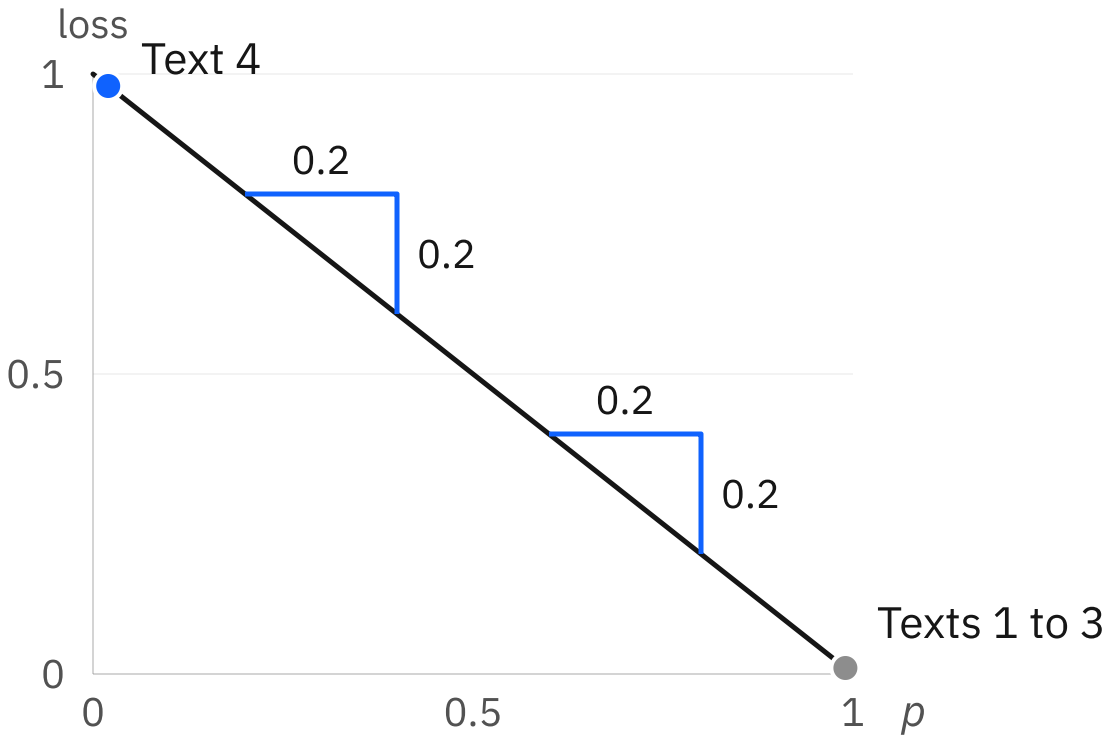
At $p = 1$ total loss $= 30$



Problem: with $1 - p$, the loss is lowest at $p = 1$, not at the honest answer $p = 0.7$. Training would make the model claim that every headline is sports.

A constant gradient slows learning from errors

	p	Loss $1-p$	Pull
Text 1	0.99	0.01	1
Text 2	0.99	0.01	1
Text 3	0.99	0.01	1
Text 4	0.02	0.98	1



Problem: with $1-p$, the mistake on Text 4 pulls no harder than a text that is already right.

Not additive over independent texts

Text 1: $p = 0.5$ | Text 2: $p = 0.5$

One text at a time

$$(1 - 0.5) + (1 - 0.5) = 1$$

Both texts as one pair

$$1 - 0.5 \times 0.5 = 0.75$$

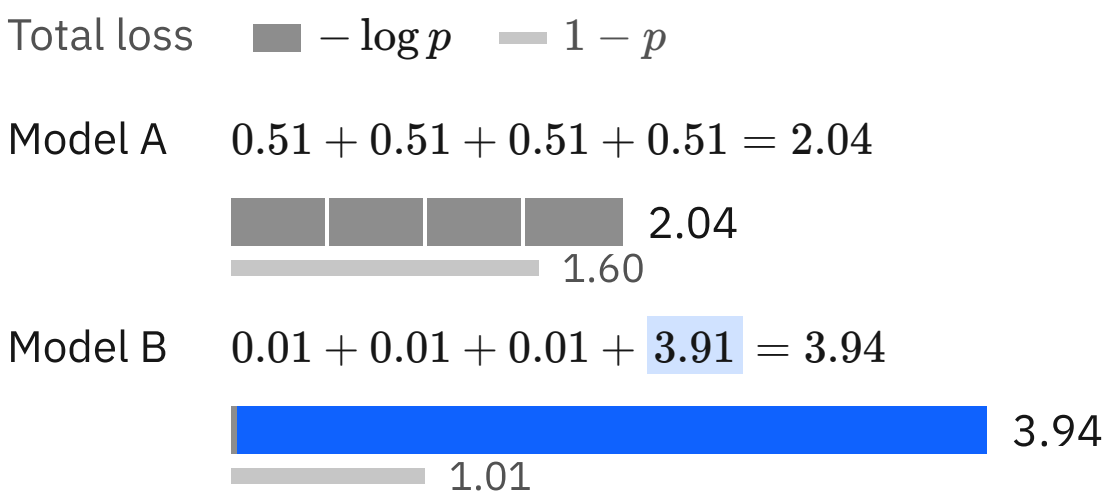
Texts	One at a time	As one group
2	1	0.750
4	2	0.938
8	4	0.996

Problem: with $1 - p$, the loss of the same texts depends on how we group them. A good loss gives the same answer either way.

A better loss function

$$L(p) = -\log p$$ log is the natural logarithm.

	Model A	Model B
Text 1	0.6	0.99
Text 2	0.6	0.99
Text 3	0.6	0.99
Text 4	0.6	0.02
Correct	4 of 4	3 of 4



Solved: $-\log p$ tells a confident error from an uncertain one. Model A, correct on every text, has the lower loss.

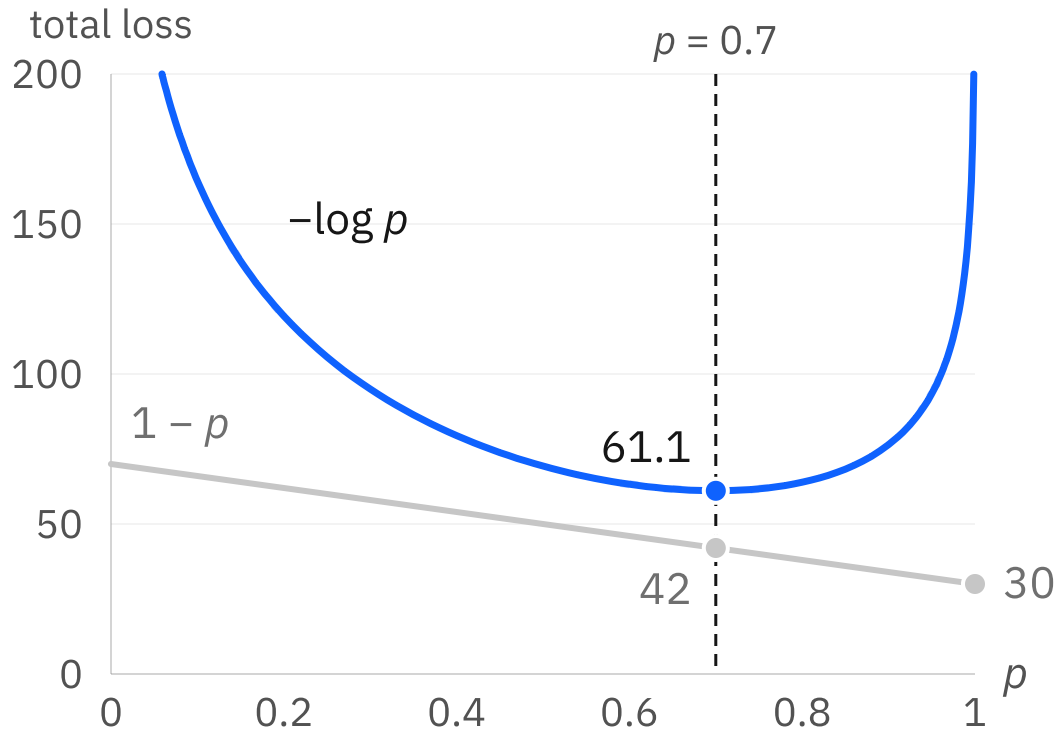
Prefers the honest answer

Headlines	$p(\text{true topic})$	Loss
■ 70 sports	p	$L(p) = -\log p$
■ 30 entertainment	$1 - p$	$-\log(1-p)$

$$\begin{aligned}\text{Total loss} &= 70\,L(p) + 30\,L(1-p) \\ &= -70\log p - 30\log(1-p)\end{aligned}$$

At $p = 0.7$ total loss $= 61.1$

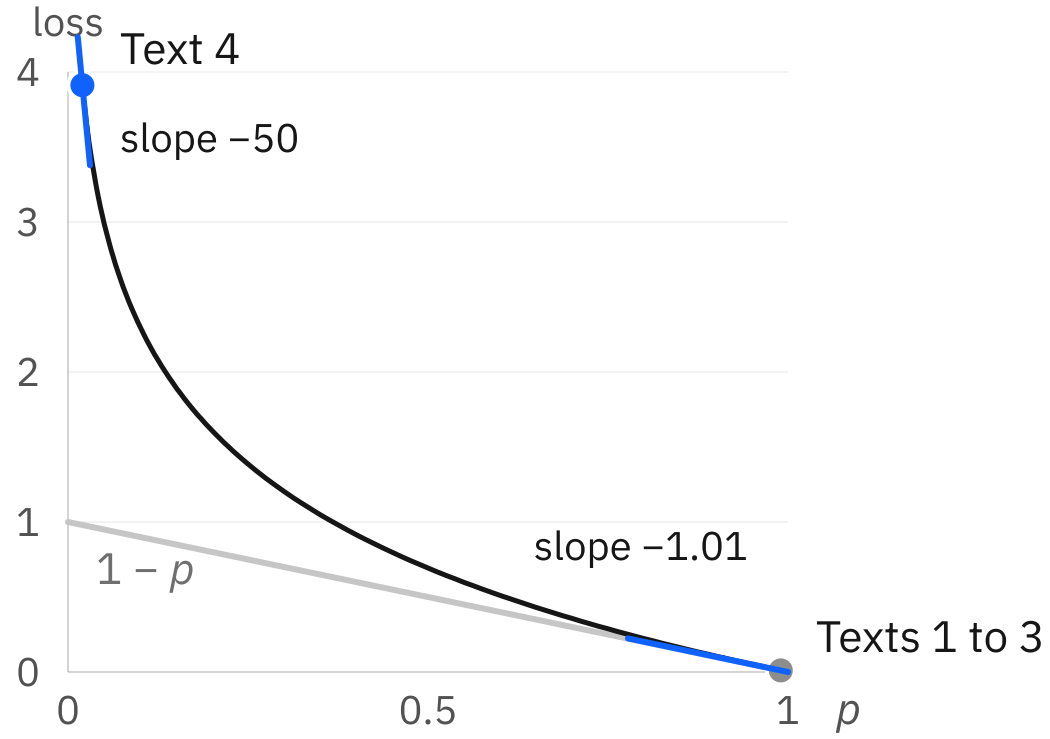
At $p = 1$ total loss $= \infty$



Solved: with $-\log p$, the total loss is lowest at the honest answer, $p = 0.7$.

A steeper gradient for larger errors

	p	**Loss** $-\log p$	**Pull**
Text 1	0.99	0.01	1.01
Text 2	0.99	0.01	1.01
Text 3	0.99	0.01	1.01
Text 4	0.02	3.91	50



Solved: with $-\log p$, the larger the error, the harder the text pulls.

Additive over independent texts

Text 1: $p = 0.5$ | Text 2: $p = 0.5$

One text at a time

$$-\log 0.5 - \log 0.5 = 1.39$$

Both texts as one pair

$$-\log(0.5 \times 0.5) = -\log 0.25 = 1.39$$

	$-\log p$		$1 - p$	
Texts	**Alone**	**Group**	**Alone**	**Group**
2	1.39	1.39	1	0.750
4	2.77	2.77	2	0.938
8	5.55	5.55	4	0.996

Solved: with $-\log p$, the losses of independent texts add up, however we group them.

One loss, several names

Negative log-likelihood	$-\log p(\text{true topic} \mid \text{text})$
Log loss	the same number, under its name in scikit-learn and on Kaggle
Over the training set	$\frac{1}{N}\sum_{n=1}^{N} -\log p(y_n \mid x_n)$
Cross-entropy	$H(q,p) = -\sum_k q_k \log p_k \;\; = -\log p_{\text{true}}$ when q is one-hot

Minimizing it is maximum likelihood. The counting in Part 2 was maximum likelihood on the words.

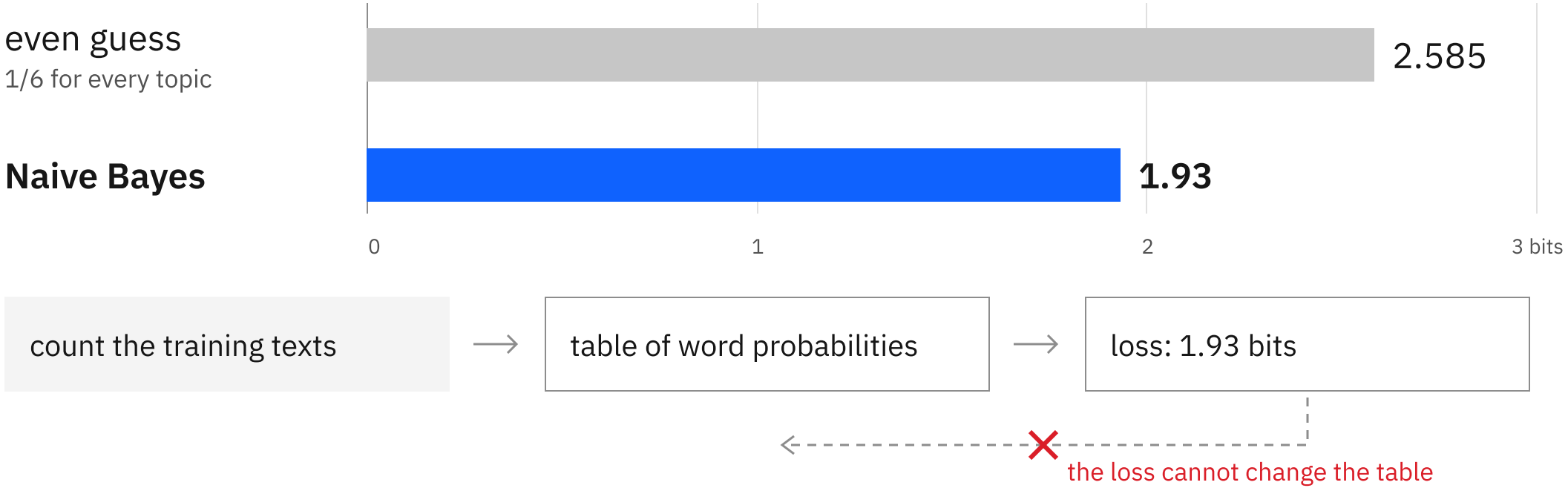
Entropy $H(q)$: the smallest possible cross-entropy, reached when $p = q$. Class 3, with perplexity.

Part 4: Logistic regression

The same softmax, with weights learned from the loss.

Naive Bayes can’t see its loss

Loss on the 4,000 dev texts, in bits per text. Lower is better.



We need a model whose parameters the loss can adjust.

The simplest learning model

$$z_k = \mathbf{w}_k \cdot \mathbf{x} + b_k$$

the score of topic *k*

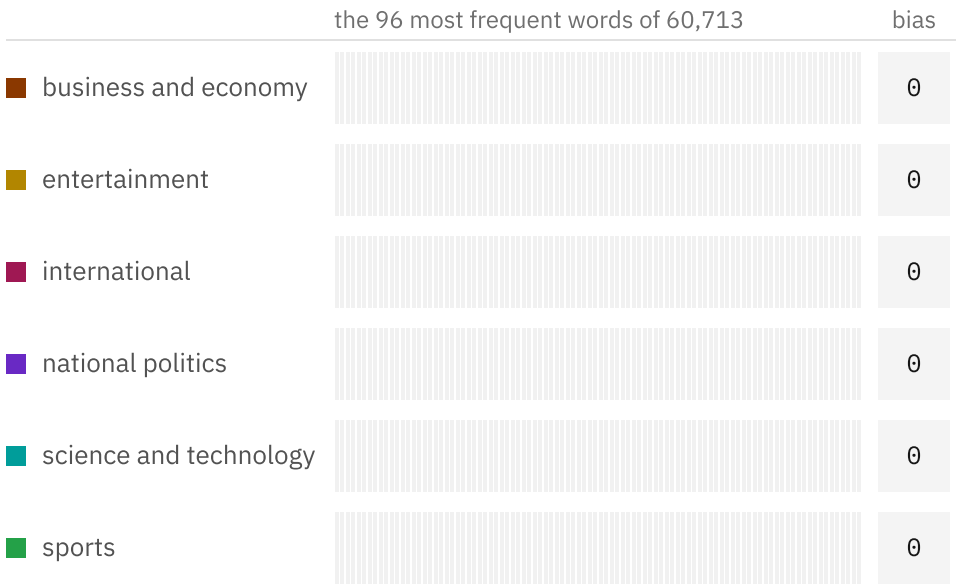
$$p(k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

softmax, as in Part 2

	Naive Bayes	Logistic regression
input $\mathbf{x}$	word counts	TF-IDF vector: any numbers
weight of word *w* for topic *k*	$\log p(w \mid k)$	any number, positive or negative
bias b_k	$\log p(k)$	any number
found by	counting	learning from the loss

The weight table

Six topics, 60,713 words: a weight for each pair, and a bias per topic. All start at 0.



The vocabulary: words seen at least twice in the 40,000 training texts.

c01_bow_lr.ipynb, cell 13, shortened

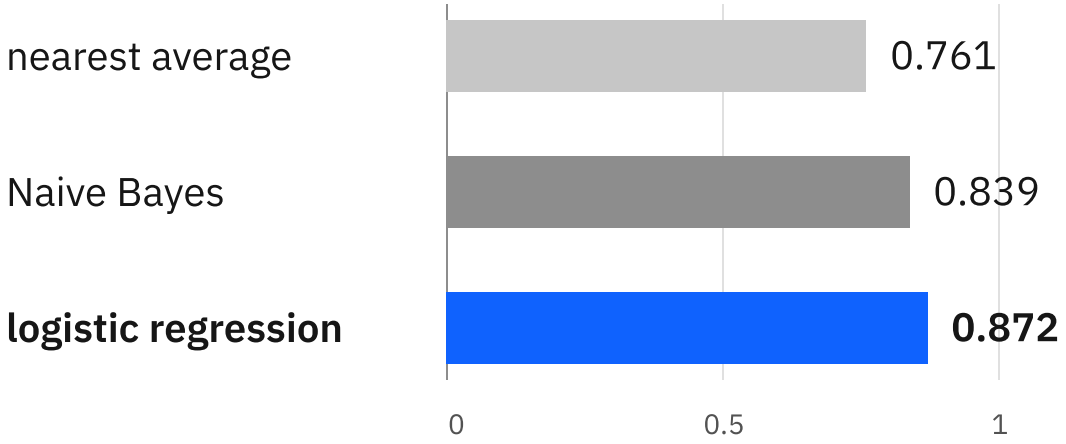
```
def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True)
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)

def fit(self, X, y):
    n, d = X.shape
    self.W = np.zeros((d, self.K))
    self.b = np.zeros(self.K)
    Y = np.eye(self.K)[y]
    for epoch in range(self.epochs):
        for idx in batches(n):
            Xb, Yb = X[idx], Y[idx]
            P = softmax(Xb @ self.W + self.b)
            loss = -np.log(P[range(len(idx)), y[idx]])
            G = (P - Yb) / len(idx)
            self.W -= self.lr * Xb.T @ G
            self.b -= self.lr * G.sum(axis=0)

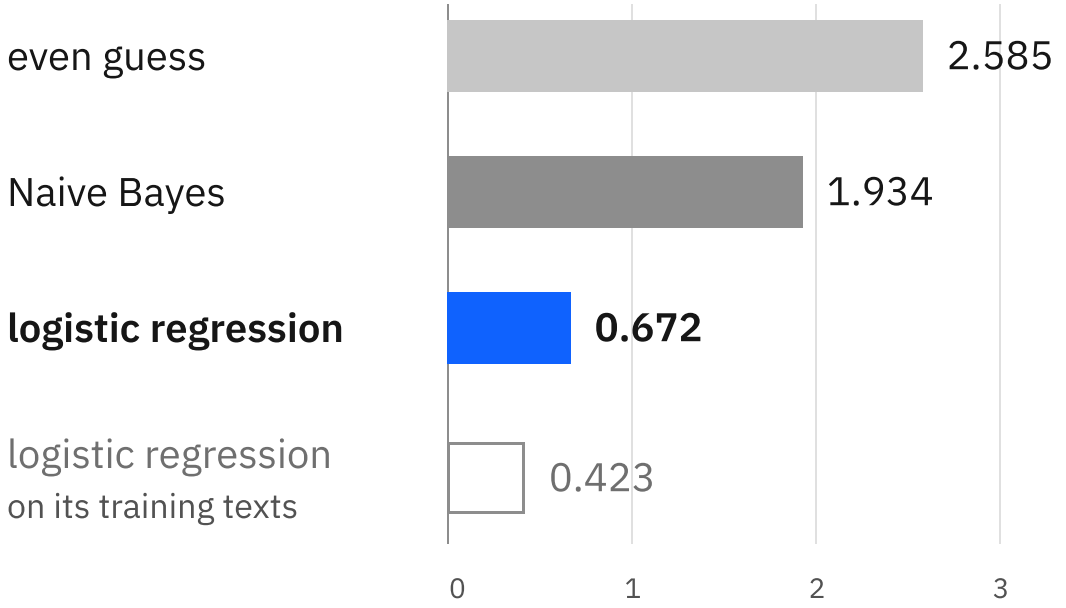
def predict_proba(self, X):
    return softmax(X @ self.W + self.b)
```

Results

Dev macro-F1, higher is better



Dev loss in bits per text, lower is better



Naive Bayes is generative: it learns how each topic writes. Logistic regression is discriminative: it learns to separate the topics from its mistakes.

Part 5: LR limitations

What logistic regression still cannot do.

Order is still missing

	bangladesh	beat	dhaka	film	india	…
Bangladesh beat India	1	1	0	0	1	…
India beat Bangladesh	1	1	0	0	1	…

The same vector, so the same six probabilities, in Naive Bayes and in logistic regression.

> Class 3
>
> A model for each topic that predicts each word from the one before it. Round R1 of the leaderboard.

What we actually want

	Naive Bayes	logistic regression	**what we want**
gives probabilities	√	√	√
learns from its mistakes	✗	√	√
can write	√	✗	√
sees word order	✗	✗	√

A language model has all four properties.

Class 3 adds order: n-gram models, by counting. Classes 5 and 6 add learning: neural language models.

References

Textbook

Jurafsky, D. and Martin, J. H. (2026). *Speech and Language Processing*, 3rd ed. draft, August 2026. Chapter 4, sections 4.1, 4.3, 4.5 and 4.7–4.9, pp. 93–115 (Naive Bayes, logistic regression, cross-entropy, training); chapter 1, section 1.2, p. 9 (classification as language modelling); chapter 3, pp. 68–88 (reading for class 3).

Data and code

Class 1 notebooks. c01_bow_lr.ipynb (logistic regression on TF-IDF, cells 1–17, seed 0) and c01_minimal.ipynb (nearest topic average), on the C1 training set (40,000 texts) and dev set (4,000 texts).

Papers

Ng, A. Y. and Jordan, M. I. (2002). *On discriminative vs. generative classifiers: a comparison of logistic regression and naive Bayes.* Advances in Neural Information Processing Systems 14.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019). *Language models are unsupervised multitask learners.* OpenAI. (GPT-2.)

Thank you.